

Chapter 3: Processes

Process Concept

- A **process** is a program in execution.
- It's more than just program code, representing a unit of work in modern time-sharing systems.
- The system consists of a collection of processes: operating system processes executing system code and user processes executing user code.
- All these processes can execute concurrently with the CPU(s) multiplexed among them.
- By switching between processes, the OS can make the computer more productive.

Multiple Parts of a Process

A process includes several parts:

- The program code, also called the **text section**
- Current activity, represented by the value of the **program counter**
- Content of **processor registers**
- **Process stack**, containing temporary data (function parameters, return addresses, local variables)
- **Data section**, containing global variables
- **Heap**, containing memory dynamically allocated during runtime

Program vs. Process

- A **program** is a passive entity stored on disk (executable file).

A file containing a list of instructions stored on a disk.

- A **process** is an active entity with a program counter specifying the next instruction to execute and a set of associated resources.

A program becomes a process when the executable file is loaded from disk into memory. Two common techniques for this include:

- Double-clicking an icon representing the executable file
- Entering the name of the executable file on the command line

It's important to note that two processes may be associated with the same program but are separate execution sequences. For example, multiple users executing the same editor program, where each one is a separate process.

Process State

As a process executes, it changes state. Each process may be in one of the following states:

- **New**: The process is being created but has not yet been admitted to the pool of executable processes by the OS.
- **Running**: Instructions are being executed.
- **Blocked/Waiting**: The process is waiting for some event to occur (I/O completion or reception of a signal).
- **Ready**: The process is waiting to be assigned to a processor.
- **Terminated**: The process has finished execution and has been released from the pool of executable processes by the OS.

New State Details

- A process that has just been created and not yet been admitted to the pool of executable processes by the OS.
- The new process has not yet been loaded into main memory, although its process control block has been created.
- Information concerning the process that is needed by the OS is maintained in control tables in main memory.
- The code of the program to be executed is not in main memory, and no space has been allocated for the data associated with that program.
- While in the New state, the program remains in secondary storage, typically disk storage.

Terminated/Exit State Details

- A process that has been released from the pool of executable processes by the OS, either because it halted or because it aborted for some reason.
- A process is terminated when it reaches a natural completion point, when it aborts due to an unrecoverable error, or when another process with the appropriate authority causes the process to abort.
- Termination moves the process to the exit state. At this point, the process is no longer eligible for execution.
- The tables and other information associated with the job are temporarily preserved by the OS, which provides time for auxiliary or support programs to extract any needed information.

Process State Transitions

- Only one process can be running on any processor at any instant, although many processes may be ready and waiting.
- **Null to New:** A new process is created to execute a program.
- **New to Ready:** The OS will move a process from the New state to the Ready state when it is prepared to take on an additional process.
- **Ready to Running:** When it is time to select a process to run, the OS chooses one of the processes in the Ready state. This is the job of the scheduler or dispatcher.
- **Running to Exit:** The currently running process is terminated by the OS if the process indicates that it has completed, or if it aborts.
- **Running to Ready:** The most common reason for this transition is that the running process has reached the maximum allowable time for uninterrupted execution.
 - This transition can also occur if the OS assigns different levels of priority to different processes.

Process Control Block (PCB)

The Process Control Block (PCB) is a repository of any information related to a process. It contains:

- **Process state:** running, waiting, etc.
- **Program counter:** address of the next instruction to be executed for this process.
- **CPU registers:** contents of all process-centric registers, which may vary in number and type depending on system architecture.
- **CPU scheduling information:** process priorities, scheduling queue pointers, any other scheduling parameters (Time Quantum).
- **Memory-management information:** memory allocated to the process, value of the Base and limit registers, the page tables or segment tables.
- **Accounting information:** CPU used, clock time elapsed since start, time limits.
- **I/O status information:** list of I/O devices allocated to process, list of open files.

Process Representation in Linux

In Linux, a process is represented by the C structure `task_struct`:

```
struct task_struct {
    pid_t          pid;           /* process identifier */
    long           state;         /* state of the process */
    unsigned int    time_slice;   /* scheduling information */
    struct task_struct *parent;   /* this process's parent */
    struct list_head children;    /* this process's children */
    struct files_struct *files;   /* list of open files */
    struct mm_struct *mm;        /* address space of this process */
};
```

Threads

What is a Thread?

A thread is a path of execution within a process. A process can contain multiple threads.

Traditional Heavy Weight Process

- Has a single thread of control.
- Performs a single thread of execution or one task at a time.
 - For example, if a process is running a word processor program, a single thread of instructions is being executed. The user could not simultaneously type in characters and run the spell checker within the same process.

Multithreaded Process

If a process has multiple threads of control, it can do more than one task at a time.

Why Multithreading?

- A thread is also known as a lightweight process.
- The idea is to achieve parallelism by dividing a process into multiple threads.

Sharing Among Threads

Threads are not independent of one another like processes. Threads share with other threads their:

- code section
- data section
- OS resources (like open files and signals)

Thread-Specific Attributes

Like a process, a thread has its own:

- program counter (PC)
- register set
- stack space

Process vs. Thread

- Threads within the same process run in a shared memory space, while processes run in separate memory spaces.
- Threads are not independent of one another like processes.

Opening a new browser (e.g., Chrome) is an example of creating a process. On the contrary, opening multiple tabs in the browser is an example of creating threads.

Advantages of Threads over Processes

1. **Responsiveness**: If the process is divided into multiple threads, if one thread completes its execution, then its output can be immediately returned.
2. **Faster context switch**: Context switch time between threads is lower compared to process context switch. Process context switching requires more overhead from the CPU.
3. **Resource sharing**: Resources like code, data, and files can be shared among all threads within a process.
 - Note: stack and registers can't be shared among the threads. Each thread has its own stack and registers.
4. **Communication**: Communication between multiple threads is easier, as the threads share a common address space, while in a process, we have to follow some specific communication techniques for communication between two processes.
5. **Effective utilization of a multiprocessor system**: If we have multiple threads in a single process, then we can schedule multiple threads on multiple processors, which will result in faster execution.

Types of Threads

There are two types of threads:

- User Level Thread
- Kernel Level Thread

User Threads

- Management is done by user-level threads library.
- Three primary thread libraries:
 - POSIX Pthreads
 - Windows threads
 - Java threads
- Implemented by a thread library at the user level.
- The library provides support for thread creation, scheduling, and management in User space.
- Generally fast to create and manage than kernel threads.
- If a thread performs a blocking system call, then any user-level thread in the application will cause the entire process to block, even if other threads are available to run within the application.

Kernel Threads

- Supported directly by the Kernel.
- Examples include virtually all general-purpose operating systems, including:
 - Windows
 - Solaris
 - Linux
 - Tru64 UNIX
 - Mac OS X
- Kernel performs support for thread creation, scheduling, and management in Kernel space with no support/no intervention from the kernel.
- Generally slower to create and manage than user threads
- If the Kernel is single-threaded, the kernel can't schedule another thread performing a blocking system call for execution.

Multithreading Models

Many systems provide support for both user and kernel threads, resulting in different multithreading models:

1. Many-to-One
2. One-to-One
3. Many-to-Many

ULT and KLT

- By default, an application begins with a single thread and begins running in that thread.
- This application and its thread are allocated to a single process managed by the kernel.
- At any time that the application is running (the process is in the Running state), the application may spawn a new thread to run within the same process.
- Spawning is done by invoking the spawn utility in the threads library.
- Control is passed to that utility by a procedure call.
- The threads library creates a data structure for the new thread and then passes control to one of the threads within this process that is in the Ready state, using some scheduling algorithm.
- When control is passed to the library, the context of the current thread is saved, and when control is passed from the library to a thread, the context of that thread is restored.
- The context essentially consists of the contents of user registers, the program counter, and stack pointers.
- All of the activity described above takes place in user space and within a single process.
- The kernel is unaware of this activity. The kernel continues to schedule the process as a unit and assigns a single execution state (Ready, Running, Blocked, etc.) to that process.

Many-to-One Model

- Many user-level threads are mapped to a single kernel thread.
- Only one thread can access the Kernel at a time.
- Thread Management is done in User space, so it is efficient.
- If one thread makes a blocking system call, the entire process will block.
- Multiple threads are unable to run in parallel on a multicore system because only one may be in the kernel at a time.
- Few systems currently use this model.
 - Examples:
 - Solaris Green Threads
 - GNU Portable Threads

: One-to-One Model

- Each user-level thread maps to a kernel thread.
- Creating a user-level thread creates a kernel thread.
- More concurrency than many-to-one.
- By allowing another thread to run when a thread makes a blocking system call.
- Allows multiple threads to run in parallel on multiprocessors.
- Creating a user thread requires creating the corresponding kernel thread.
- The overhead of creating kernel threads can burden the performance of an application.
- The number of threads per process is sometimes restricted due to overhead.
- Examples:
 - Windows
 - Linux
 - Solaris 9 and later

M:N Many-to-Many Model

- Allows many user-level threads to be mapped to many kernel threads.
- Allows the operating system to create a sufficient number of kernel threads.
- The number of kernel threads may be specific to either a particular application or a particular machine.
- Examples:
 - Solaris prior to version 9
 - Windows with the ThreadFiber package

Drawbacks of Many-to-One Model

- Allows the developer to create as many user threads as needed.
- True concurrency is not gained because the kernel can schedule only one thread at a time.

Drawbacks of One-to-One Model

- Allows for greater concurrency, but the developer has to be careful not to create too many threads within an application.

Advantages of Many-to-Many Model

- Overcomes these shortcomings.
- Developers can create as many user threads as necessary.
- Corresponding Kernel threads can run in parallel on a multiprocessor.
- When a thread performs a blocking system call, the kernel can schedule another thread for execution.

User Level Thread vs. Kernel Level Thread

Feature	User Level Thread	Kernel Level Thread
Management	Managed by user-level threads library.	Supported directly by the Kernel.
Implementation	Implemented by a thread library at the user level.	Supported directly by the OS.
Scheduling & Mgmt	Library provides support for thread creation, scheduling, and management in User space.	Kernel performs support for thread creation, scheduling, and management in Kernel space with no support/no intervention.
Creation Speed	Generally fast to create and manage.	Generally slower to create and manage.
Blocking System Calls	If a thread performs a blocking system call, the entire process will block.	If the Kernel is single-threaded, the kernel can't schedule another thread performing a blocking system call for execution.
Examples	POSIX Pthreads, Windows threads, Java threads	Windows, Solaris, Linux, Tru64 UNIX, Mac OS X

Interprocess Communication (IPC)

- Cooperating processes need an interprocess communication (IPC) mechanism to exchange data and information.
- Two models of IPC:
 - Shared memory
 - Message passing

Process Scheduling

- Maximize CPU use, quickly switch processes onto CPU for time sharing.
- The process scheduler selects among available processes for the next execution on the CPU.
- Maintains scheduling queues of processes:
 - Job queue
 - Ready queue
 - Device queues
- Processes migrate among the various queues.

Scheduling Queues

- **Job queue:** Each new process goes into the job queue. Processes in the job queue reside on mass storage and await the allocation of main memory. As a process enters the system, they are put on the job queue.
- **Ready queue:** Set of all processes residing in main memory, ready and waiting to execute. The set of all processes that are in main memory and are waiting for CPU time are kept in the ready queue. Generally stored using a linked list.
- **Device queues:** Set of processes waiting for an I/O device. Each device has its own device queue.

Queue Summary

Queue	Description
Job Queue	Each new process goes into the job queue. Processes in the job queue reside on mass storage and await the allocation of main memory.
Ready Queue	The set of all processes that are in main memory and are waiting for CPU time is kept in the ready queue.
Waiting Queues	The set of processes waiting for allocation of certain I/O devices is kept in the waiting (device) queue.

A new process is initially put in the ready queue and waits there to be dispatched. Once assigned to the CPU and is executing, one of several events may occur:

- The process could issue an I/O request and be placed in the I/O queue.
- The process could create a new subprocess and wait for its termination.
- The process could be removed forcibly from the CPU due to an interrupt and put back in the ready queue.

Finally, when the process terminates, it is removed from all queues, and PCBs and resources are deallocated.

Schedulers

The OS must select processes from these queues in some fashion. The selection process is carried out by the appropriate scheduler. The schedulers are:

- Long-term scheduler (or job scheduler)
- Short-term scheduler (or CPU scheduler)
- Medium-term scheduler

Long-Term Scheduler

- Also known as the Job Scheduler.
- Selects which processes should be brought into the ready queue.
- Changes the process state from New to Ready (i.e., From Disk to Main Memory).
- Invoked infrequently (seconds, minutes) and may be slow.
- Controls the degree of multiprogramming (i.e., the number of processes in memory).
- On some systems, the long-term scheduler may be absent or minimal.
 - Time-sharing systems like UNIX often have no LTS. The stability of such systems depends on physical limitations such as the number of available terminals or the self-adjusting nature of human users.
- Processes can be described as either:
 - **I/O-bound process**: spends more time doing I/O than computations, many short CPU bursts.
 - **CPU-bound process**: spends more time doing computations; generates I/O requests infrequently, few very long CPU bursts.

The long-term scheduler strives for a good process mix of both I/O-bound and CPU-bound processes.

- If all Processes are I/O bound, the ready queue will almost always be empty, and the STS will have little to do, leading to CPU underutilization.
- If all Processes are CPU bound, the I/O waiting queue will almost always be empty, and devices will go unused.

Short-Term Scheduler

- Selects processes from the ready queue and allocates the CPU for execution.
- Sometimes the only scheduler in a system.
- Invoked frequently (milliseconds).
- Also known as the CPU scheduler, as it selects a process from the ready queue and yields control of the CPU to the process.
- Must be fast, as a process may execute for only a few milliseconds before waiting for an I/O request, so the STS needs to select another process from the ready queue. Often, the STS executes at least once every 100 milliseconds.

Medium-Term Scheduling

- Can be added if the degree of multiple programming needs to decrease.
- Removes a process from memory, stores it on disk, and brings it back in from disk to continue execution (swapping).
- Why?
 - To improve the process mix.
 - Memory requirements have overcommitted available memory, requiring memory to be freed up.

Context Switch

- The context of a process is represented in the PCB.
- When the CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a context switch.
- The kernel saves the context of the old process in its PCB and loads the saved contents of the new process from its PCB to run.
- Context-switch time is overhead; the system does no useful work while switching. Varies from machine to machine.
- The more complex the OS and the PCB, the longer the context switch.
- Time is dependent on hardware support. Some hardware provides multiple sets of registers per CPU with multiple contexts loaded at once, which involves changing the pointer to the current register set.

Operations on Processes

The system must provide mechanisms for:

- process creation
- process termination

Process Creation

- A parent process creates child processes, which, in turn, create other processes, forming a tree of processes.
- Generally, processes are identified and managed via a process identifier (pid).
- Resource sharing options (CPU time, memory, files, I/O devices):
 - Parent and children share all resources.
 - Children share a subset of parents' resources.
 - Parent and child share no resources (child obtains resources directly from OS).
- Restricting a child process to a subset of parents' resources prevents any process from overloading the system by creating too many subprocesses.
- Execution options:
 - Parent and children execute concurrently.
 - Parent waits until children terminate.
- In terms of Address space:
 - Child duplicate of parent.
 - Child has a separate program loaded into it.
- `fork()` creates a new process by duplicating the calling process. The new process, referred to as the child, is an exact duplicate of the calling process, referred to as the parent.
- The `exec()` family of functions replaces the current process image with a new process image. It loads the program into the current process space and runs it from the entry point.

UNIX examples

- `fork()` system call creates a new process
- `exec()` system call used after a `fork()` to replace the process's memory space with a new program

`fork` starts a new process which is a copy of the one that calls it, while `exec` replaces the current process image with another (different) one.

The `execvp` function is most commonly used to overlay a process image that has been created by a call to the `fork` function. `execvp` replaces the calling process image with a new process image. This has the effect of running a new program with the process ID of the calling process. Note that a new process is not started; the new process image simply overlays the original process image.

The `exec()` family consists of the following functions:

```
int execl(const char *path, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
int execl(const char *path, const char *arg, ..., char *const envp[]);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char*const envp[]);
```

The `execvp` function prototype:

```
int execvp(const char *path, const char *arg0, ..., NULL);
```

- `path` identifies the location of the new process image within the hierarchical file system (HFS).
- `arg0, ..., NULL` is a variable-length list of arguments that are passed to the new process image.
- Each argument is specified as a null-terminated string, and the list must end with a NULL pointer. The first argument, `arg0`, is required and must contain the name of the executable file for the new process image.

A successful call to `execvp` does not have a return value because the new process image overlays the calling process image. However, -1 is returned if the call to `execvp` is unsuccessful.

`fork()` is used for creating a new process, which is called the child process, which runs concurrently with the process that makes the `fork()` call (parent process).

It takes no parameters and returns an integer value:

- **Negative Value:** Creation of a child process was unsuccessful.
- **Zero:** Returned to the newly created child process.
- **Positive value:** Returned to the parent or caller. The value contains the process ID of the newly created child process.

Points to Note:

- The parent can create more children, or if it has nothing else to do while the child runs, it can issue a wait system call to move itself off the ready queue until the termination of the child.
- The parent process and child process are running the same program, but it does not mean they are identical.
- The OS allocates different data and states for these two processes, and the control flow of these processes can be different.

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    // make two processes which run the same program after this instruction
    fork();
    printf("Hello world!\n");
    return 0;
}
```

Output:

```
Hello world!
Hello world!
```

After a new child process is created, both processes will execute the next instruction following the `fork()` system call.

Consider the following C program:

```
main() {
    fork();
    fork();
    printf ("yes");
}
```

If we execute this code segment, how many times will the string `yes` be printed? The answer is 4.

Explanation: The number of times YES is printed is equal to the number of processes created.

Total Number of Processes = 2^n where n is the number of fork system calls.

So here $n = 2$, $2^2 = 4$. So, there are a total of 4 processes (3 new child processes and one original process).

Consider the following C program:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

void forkexample() {
    // child process because return value zero
    if (fork() == 0)
        printf("Hello from Child!\n");
    // parent process because return value non-zero.
    else
        printf("Hello from Parent!\n");
}

int main() {
    forkexample();
    return 0;
}
```

Output:

```
Hello from Child!
Hello from Parent!
```

OR

```
Hello from Parent!
Hello from Child!
```

Here, two outputs are possible because the parent process and child process are running concurrently. So, we don't know whether the OS will first give control to the parent process or the child process.

Consider the following C program:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

void forkexample() {
    int x = 1;
    if (fork() == 0)
        printf("Child has x = %d\n", ++x);
    else
        printf("Parent has x = %d\n", --x);
}

int main() {
    forkexample();
    return 0;
}
```

Output:

```
Parent has x = 0
Child has x = 2
```

OR

```
Child has x = 2
Parent has x = 0
```

Here, the global variable change in one process does not affect the other process because each process has its own data and states.